

Programmation procédurale : bases



On parle de quoi ?

1. [Fonctions principales](#)
2. [Commentaires](#)
3. [Impression](#)
4. [Variable](#)
5. [Impression \(suite\)](#)
6. [Opérateurs](#)
7. [Lecture](#)
8. [Exercices de synthèse](#)

Fonction principale

— * Fonction principale

instruction1
instruction2
instruction3
...

```
void main(void){  
  
    instruction1  
    instruction2  
    instruction3  
  
}
```

- **main** est le **point d'entrée** du programme
- **instruction1** est la première instruction (le programme commence)
- **instruction3** est la dernière instruction (le programme termine)

Commentaires

Sur une seule ligne

```
// Déclaration et affectation en C  
int a; // déclaration d'une variable de type entière  
a = 3 // affectation
```

Sur plusieurs lignes

```
/*  
    Programme calculant  
    la moyenne d'une classe  
*/  
void main(void)
```

Impression

L'impression sur l'écran se fait via la fonction `printf` ou `printf_s`

```
#include <stdio.h>

void main(void){
    printf("Bonjour !");
    printf_s("Bonjour !");
}
```

→ `stdio.h` est la librairie standard pour les entrées/sorties

Variables

Exercice : combien y a-t-il de variable ?

```
└── * Calcul montant commande  
| obtenir quantite  
| montant = quantite * 2.5  
| sortir montant  
└──────────────────
```

Variables

Une **variable** est une **zone mémoire** qui **stocke de l'information**.

Une variable est **déclarée** en spécifiant :

1. son type
2. son nom
3. sa valeur (via une affectation)

→ la valeur peut être spécifiée en même temps que la déclaration ou après (mais pas avant !).

Déclaration d'une variable : le nom

Le nom d'une variable :

- peut être constitué de lettres, chiffres et/ou _
- commence par une lettre ou _
- est "case sensitive" : distinction entre une minuscule et une majuscule

Bonnes pratiques :

- le nom d'une variable ne commence jamais par une majuscule
- écriture en "camelCase"

Exercices : quels sont les noms corrects/incorrects ?

- Chif Affaire
- _ChifAffaire
- 1NbEtudiant
- chifAffaire1
- chifaffaire

Déclaration d'une variable : le nom



snake_case

Pros: Concise when it consists of a few words.
Cons: Redundant as hell when it gets longer.
`push_something_to_first_queue, pop_what, get_whatever...`



PascalCase

Pros: Seems neat.
`GetItem, SetItem, Convert, ...`
Cons: Barely used. (why?)



camelCase

Pros: Widely used in the programmer community.
Cons: Looks ugly when a few methods are n-worded.
`push, reserve, beginBuilding, ...`



skewer-case

Pros: Easy to type.
`easier-than-capitals, easier-than-underscore, ...`
Cons: Any sane language freaks out when you try it.



SCREAMING_SNAKE_CASE

Pros: Can demonstrate your anger with text.
Cons: Makes your eyes deaf.
`LOOK_AT_THIS, LOOK_AT_THAT, LOOK_HERE_YOU_MORON, ...`



nocase

Pros: Looks professional.
Cons: Misleading af.
`supersexyhippocampus, bool penisbig, ...`



fUcKtHeCaSe

Pros: Can live outside of the law.
Cons: Can be out of a job.



SPOngeBob CaSE

Pros: can mock your colleague for choosing a stupid variable name
Cons: you're really unlikeable

Déclaration d'une variable : le type

En c, il existe 4 grands types, eux-même constitués de variants :

Le type entier

- `short int` ou `short` (2 octets)
- `int` (4 octets)
- `long int` ou `long` (4 octets)

Le type réel

- `float` (4 octets)
- `double` (8 octets)
- `long double` (16 octets)

Le type caractère

- `char` (1 octet)

Le type booléen (dans `stdbool.h`)

- `bool` (`true` / `false`)

Déclaration d'une variable : le type

```
int nbEtudiants;      // variable de type entier  
double pourcentageMoyen; // variable de type réel  
char finPhrase;      // variable de type caractère  
bool estComplet;     // variable de type booléen
```

Déclaration d'une variable : la valeur

Lorsqu'on "donne" une valeur à une variable, on parle **d'affectation** ou **d'assignation**.

Une affectation/assignation peut se faire :

- en même temps que la déclaration

```
int nbEtudiants = 20;  
double pourcentageMoyen = 0.2;  
char finPhrase = 'c';  
bool estComplet = false;
```

- dans la suite du programme

```
int nbEtudiants;  
double pourcentageMoyen;  
char finPhrase;  
bool estComplet;  
nbEtudiants = 20;  
pourcentageMoyen = 0.2;  
finPhrase = 'c';  
estComplet = false;
```

Déclaration d'une constante

Une **constante** est une variable dont sa **valeur ne peut pas changer** pendant toute l'exécution du programme.

→ on utilise le mot-clé **const**

→ toute affectation après la déclaration sera rejetée par le compilateur.

```
const int n = 20; // n contient la valeur 20  
n = 10;          // erreur !
```

```
const int n = 20; // n contient la valeur 20  
const int n = 10; // erreur !
```

Affectation

Une affectation a toujours la forme `maVariable = monExpression`

Une expression peut prendre plusieurs formes :

- une valeur

```
int maVariable = 1;
```

- une opération

```
int maVariable = 10 + 15;
```

- une variable

```
int maVariable = 1;
```

```
int monAutreVariable = 2;
```

```
maVariable = monAutreVariable;
```

```
maVariable = monAutreVariable + 5;
```

Impression (suite)

```
int nbInscrits = 100;  
printf("Nombre d'inscrits = %d", nbInscrits);
```

```
double longueur = 5.5;  
printf("Perimetre du carré : %.2f m", 4 * longueur);
```

```
int nbInscritsInfo = 100;  
int nbInscritsSciences = 150;  
  
printf("Nombre d'inscrits en info : %d\n", nbInscritsInfo);  
printf("Nombre d'inscrits en sciences : %d\n", nbInscritsSciences);
```

```
int nbInscritsInfo = 100;  
int nbInscritsSciences = 150;  
  
printf("Nombre d'inscrits en info : %d\nNombre d'inscrits en sciences : %d\n", nbInscritsInfo, nbInscri
```

Impression (suite)

Type	Code de formatage
short	%hd
int	%d
long	%ld
float	%f
double	%lf
long double	%Lf
char	%c

Action	Ordre de contrôle
passage à la ligne	\n
tabulation	\t
bip sonore	\a

Affichage	Pattern
\	\\
%	%%
"	\"

Opérateurs

Opérateurs arithmétiques

- Addition : +

```
int maVariable;  
int monAutreVariable = 2;  
  
maVariable = monAutreVariable + 5;
```

- Soustraction : -

```
int maVariable = 1 - 2;
```

- Multiplication : *

```
int maVariable;  
int monAutreVariable = 2;  
  
maVariable = monAutreVariable * 2;
```

Opérateurs arithmétiques

- Division : /

```
int maVariable;  
int monAutreVariable = 2;  
  
maVariable = monAutreVariable / 2;
```

- Modulo (reste de la division d'un entier par un entier) : %

```
int nombre = 15;  
int reste;  
  
reste = 15 % 2; // reste contient la valeur 1  
reste = 15 % 3; // reste contient la valeur 0  
reste = 15 % 4; // reste contient la valeur 3
```

Type des variables en cas de division

Quelle sera la sortie ?

```
int nb1, nb2, resultatDivisionEntiere;  
double resultatDivisionReelle;  
  
nb1 = 100;  
nb2 = 30;  
  
resultatDivisionReelle = nb1/nb2;  
printf("%lf",resultatDivisionReelle);
```

Type des variables en cas de division

En C, l'opérateur de division appliqué à deux entiers, exécute la division entière :

```
double a = 1 / 3 // a contient 0 (partie entière de 0.33333...)
```

Pour "forcer" la division avec plus de précision, il faut utiliser le **casting** :

```
double a = (double) 1 / 3 // a contient 0.33333...
```

Incrémentation

- Post-incrémentation

```
int n = 1;  
n++;
```

- Pré-incrémentation

```
int n = 1;  
++n;
```

Quelle sera la sortie ?

```
int nombre = 10;  
int nombrePlusUn;
```

```
nombrePlusUn = ++nombre;  
printf("nombre : %d\nnombrePlusUn :%d\n", nombre, nombrePlusUn);
```

```
nombrePlusUn = nombre++;  
printf("nombre : %d\nnombrePlusUn :%d\n", nombre, nombrePlusUn);
```

Décrémentation

- Post-décrémentation

```
int n = 1;  
n--;
```

- Pré-décrémentation

```
int n = 1;  
--n;
```

Quelle sera la sortie ?

```
int nombre = 10;  
int nombreMoinsUn;
```

```
nombreMoinsUn = --nombre;  
printf("nombre : %d\nnombreMoinsUn :%d\n", nombre, nombreMoinsUn);
```

```
nombreMoinsUn = nombre--;  
printf("nombre : %d\nnombreMoinsUn :%d\n", nombre, nombreMoinsUn);
```

Lecture

La lecture se fait via le clavier en utilisant la fonction `scanf` ou `scanf_s` qui se trouvent dans la librairie `stdio.h`.

```
int nbEtudiants;  
scanf_s("%d", &nbEtudiants);
```

```
int nbEtudiants;  
printf("Entrez le nombre d'inscrits : ");  
scanf_s("%d", &nbEtudiants);
```

```
int abscisse, ordonnee;  
printf("Entrez les coordonnées (x,y) :");  
scanf_s("(%d,%d)", &abscisse, &ordonnee);
```

→ Ne pas oublier l'opérateur d'adressage `&` !

Exercices de synthèse

Créez un programme qui demande à l'utilisateur d'entrer le nombre d'élèves inscrits dans la section informatique. Le programme affichera ensuite : "Il y a X inscrits en informatique".

Exercices de synthèse

Créez un programme qui demande à l'utilisateur d'entrer le nombre d'élèves inscrits dans la section informatique. Le programme affichera ensuite : "Il y a X inscrits en informatique".

```
#include <stdio.h>

void main (void){
    int nbEleves;

    printf("Nombre d'élèves en informatique : ");
    scanf("%d", &nbEleves);
    printf("Il y a %d inscrits en informatique",nbEleves);
    system("pause");
}
```

Exercices de synthèse

Corrigez les erreurs du programme suivant.

```
Void main (double){  
    int nombre  
    int nombredouble;  
    printf("Nombre :");  
    scanf("%lf", nombre);  
    nombreDouble = 2 * Nombre;  
    printf("Double du nombre :", nombreDouble);  
}
```